Report
CS4303 P2 - AI
Student NO: 200007413

1 Overview & and Design

This practical implements a 2.5D round-based fighting game with enemy AI that changes based on the game state.

The 2.D nature of the game results objects defining shapes as width, height, and depth. However, the depth primarily impacts the collision detection, allowing some 2D collisions to be ignored, giving the illusion of a 3D environment.

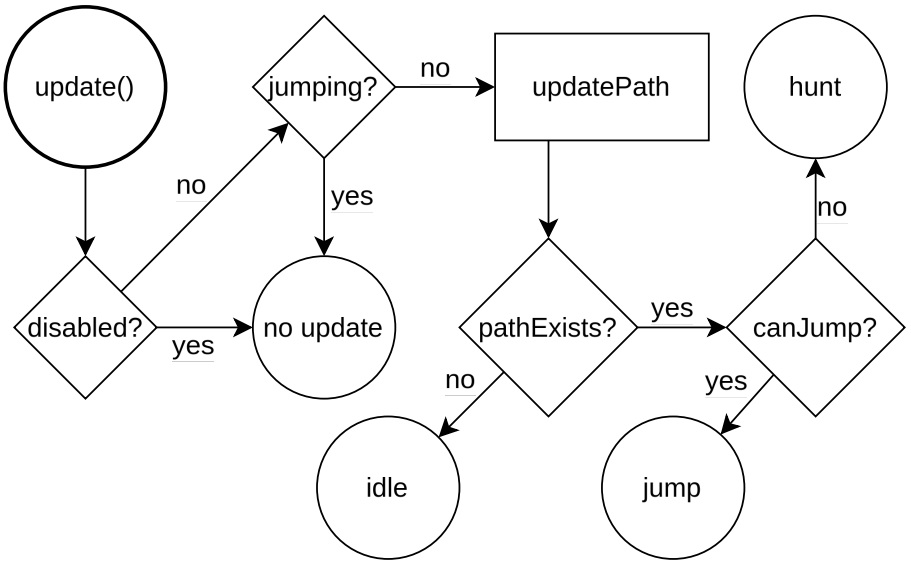

Figure 1: Behaviour tree for blob enemies

The enemy AI is designed as a decision tree, and is based upon states. Behaviour drives the movement, and updates the direction and speed of the enemy movement. Each update cycle, if the enemy is disabled, the previous movement continues to be applied. Next, the same applies if the enemy is already jumping. Otherwise, the path is updated, and if a path is not successfully found, the enemy idles, setting the direction to 0. If a path is found, then the enemy checks to see if the jump cool-down is in effect, and goes into hunting mode if it is, or jumps if not.

Hunting sets the direction to follow the path - the details of which are outlined in the following section. The hunting speed is set to the default speed of the enemy. Jumping sets the direction to point towards the player directly, and the speed is set to a higher

jump speed.

Enemies are disabled when damaged by the player's weapon, upon which they become invulnerable for a set amount of time. Enemies are re-enabled after a set time, and become vulnerable after a shorter amount of time.

2 Technical Description

In line with the underlying physics engine where movement is handled by applying forces to objects, enemies create their own `MovementForce` object, which applies a set force in a set direction each draw cycle. The force is always applied until changed.

2.1 Following Paths

A technical challenge is how to follow the generated path, which is just represented by an array of tile indices. Additionally, the path can be updated between each update cycle.

The approach taken in this submission is to take the tiles in the path in order of increasing distance from the enemy, and moving in a straight line towards that tile. When the enemy position is close enough to that tile, the tile can be removed from the path. This works with path updates, as the updated path will never include removed tiles unless the player position has changed dramatically enough to change the path completely.

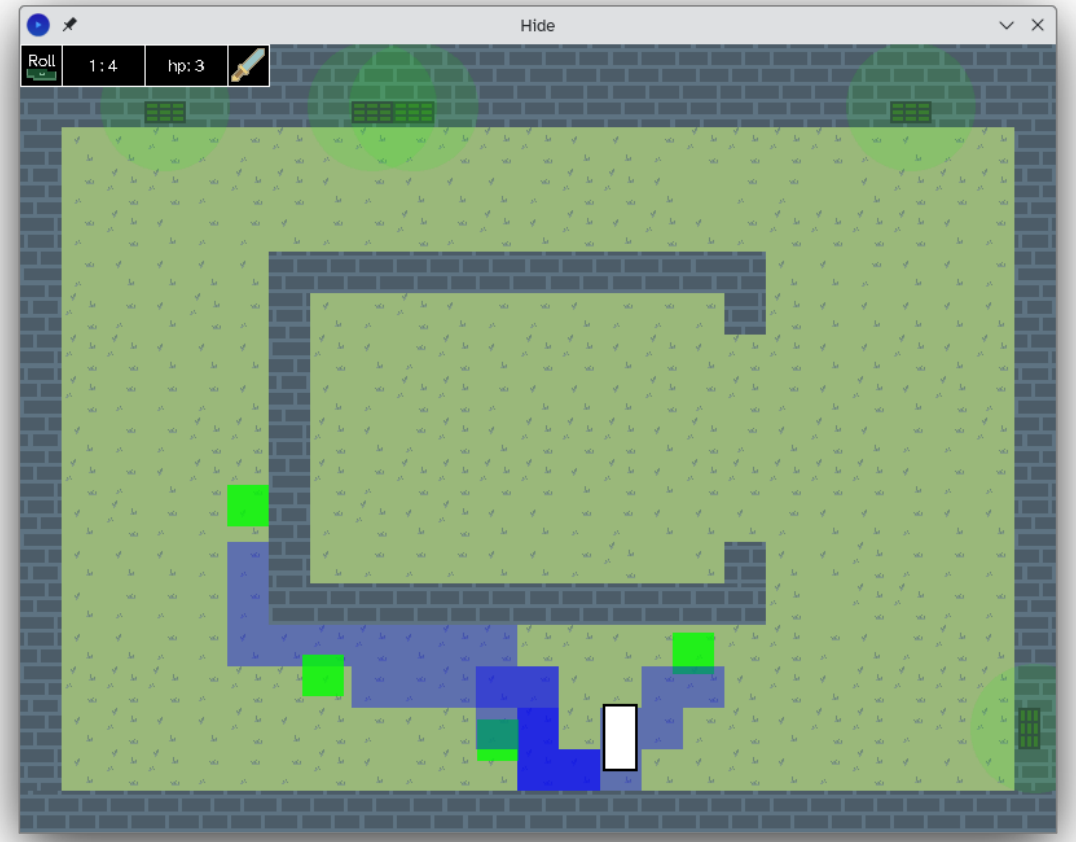

Figure 2: Routes generated from the path finder algorithm where each tile along the path is highlighted blue.

Some paths generated are shown in Figure 2. One of the difficulties encountered is in determining the start and end tiles in the search. As the player and enemy positions are continuous values in the x and y space, and not snapped to tiles, it is essential that these positions are calculated accurately.

Initially, the position was floored to the nearest integer value, as the origin of the player and enemies is set at the baseline. However, when either were contacting a wall below them, the tile value set to the tile below, which was a wall. This lead to calculating paths through walls, instead of going around them.

The solution to this problem was to subtract half of the height from the position of the enemies, and subtracting 1 from the player's y coordinate. For the enemies, their height equal to their depth, and so their x,y coordinates will always match the tile they are in. However, the player's contact region is small to allow for the 2.5D effect.

2.2 Generating Routes

Without any obstacles, the ideal path for enemies to take would be a straight line towards the player. However, with walls that cannot be walked through, A* allows the shortest path to be found taking into consideration these barriers.

From the start node, each adjacent node is added to a TreeSet openSet. A TreeSet was chosen as each node x,y is unique (tiles), and can act as a priority queue, with better performance in Java. Nodes are ordered by their total fCost, with tie breaks using their hCost. The gCost is calculated as the total distance up until that point, and the hCost is the euclidian distance to the target. This ensures a best-first search. A TreeSet allows finding the lowest cost to be found in $\mathcal{O}(1)$ time.

A Set is used as the closedSet. If the next lowest cost is in the closed set, that node is removed and skipped. Otherwise, the adjacent tiles not in the closed set are added to the openSet, marking the current node as it's parent, and we repeat until the target tile is found.

Afterwards, we start from the target node, and fetch the parent repeatedly until the source node is reached. This sequence gives the path order, and we can add the nodes as we pass along the grid to a queue representing the route.

Given the route is generated starting with the goal tile to the start tile, and the enemy follows the reverse order, a Queue data structure was used. This allows the route generator to add tiles to the end of the queue, and the enemy to read from the start of the queue in $\mathcal{O}(1)$ time.

To improve the enemy intelligence, allowing enemies to block paths together, and not bunch up, the path finding was extended to take into consideration the position of other blobs. Each update cycle, the tile position of each blob is retrieved, and tiles are weighted with higher gCosts when a blob exists on that tile. This means blobs further away from the player will run alongside other blobs instead of following the same path.

2.3 Jumping

Jumping occurs over multiple update cycles, and during this time the blob cannot change direction or force. This is achieved through not changing the movement state of the object if already jumping, and so the jump force is set once upon initial jumping, and then not changed until the jump time ends.

The movement of a jump is applied through an increase in the movement force, and setting the direction directly towards the player. One problem faced was getting the enemy to jump towards the player to maximize the likelihood of hitting them. As the player and blob positions are stored at the bottom center of the entity, using the direction between these points often leads to under-shooting the player's y-direction.

A solution is to aim for the center of the player's contact bounding box, from the center

of the blob's contact bounding box. These are calculated separately by the player and blob.

2.4 Enemy Damage

Damage checks are done through collision checks every draw cycle. When the player is damaged by an enemy, there is no update to the position of the enemy or player, as moving the player reduces the sense of control the user has over the player, and moving the enemy would often result in trapping the player where there are many enemies which felt unreasonably difficult. Instead, an invulnerability timer was added to the player, giving the user a chance to move away from the enemies.

Additionally, invulnerability upon damage was added to the enemies upon taking damage from the player's weapon, as the enemy has a knock-back force applied when they are hit. However, to simulate swinging in an arc which takes time, weapon attacks are enabled for more than one draw cycle, and so it is possible for an enemy to be hit in multiple draw cycles by the same attack. This is not desirable, as a single swing of a weapon should only be able to hit something once. Therefore, invulnerability was added to the enemies upon taking damage for a short period, giving them time for the knock-back force to move them out of range of the attack.

2.5 Blob Spawning

As an extension, blob spawning and rounds were introduced to manage the game's pace. This is handled through the `SpawnManager` class. This is initialized with an base enemies value, and a scaling rate. `Spawners` are initialized with a tile position and spawn position, allowing them to be displayed on walls, but spawn on non-blocked tiles. The `SpawnManager` randomly assigns enemies to spawn in each `Spawner` added, and tracks the total enemies spawned and killed. When the current round total enemies reaches zero, a new round is started, with a greater number of enemies to spawn.

A set spawn delay on each spawner and the spawn manager manage the pacing of enemy spawns. The increase in total enemies uses an average of a logarithmic scale and exponential scale based on the round number. This ensures that enemy numbers increase, without too quickly escalating.

3 Conclusion & Critical Analysis

The aim of this practical was to explore enemy AI, specifically with the objective to create robust and complex steering behaviours and decision trees to create fun and consistent behaviours.

This game successfully implements enemy AI with various movement and action types, whose behaviour is determined by a variety of factors in the game and enemy state.

More complex movement behaviour using the A* routing algorithm enables enemies to track and consistently reach the player and desired locations, increasing the challenge of the game.

Further development could make enemy movement smoother. As tiles are generally the same size as the enemy, the generated route represents turns with three tiles: one before the turn is made, the corner, and then the tile in the next direction. However, if there are not walls in the way, this second tile (the corner), could be skipped, shortening the overall route.

One way to solve this problem would be to look ahead at the next two tiles in the path, and have their positions influence the movement direction. This would lead to smooth turning, instead of the sharp turns made in the current implementation. Blocked tiles could also be checked to have a repulsion force, preventing turning into walls with this smoothing.

Finally, additional success was found in the implementation of spawning mechanics, modulating the pace of the game through an increasingly difficult round system.